



Département mathématiques et informatique
UFR des Sciences

TP5

“Fonction d’ordre supérieur et système de typage type”

Exercice 1 : (filter)

1. En utilisant les fonctions de l’exercice 1 du TP4 (points 1 et 6), définir une fonction `estAlpha :: Char -> Bool` qui vérifie si un caractère donné est dans l’alphabet.

Proposer une autre définition utilisant que la fonction standard `elem` d’Haskell :

```
Prelude> elem 2 [1,3]
False
Prelude> elem 2 [1,3,2]
True
```

2. En utilisant la fonction d’ordre supérieur `filter`, proposer des définitions pour les fonctions suivantes :
 - (a) une fonction `filtreAlphas :: String -> String` qui enlève tous les caractères non-alphabétiques d’une chaîne de caractères en utilisant la fonction précédente `estAlpha`.
 - (b) `rmChar :: Char -> String -> String` qui supprime toutes les occurrences d’un caractère d’une chaîne de caractères.
 - (c) `auDessus :: Int -> [Int] -> [Int]` qui supprime tous les nombres ($\leq$) au nombre donné en entrée. Proposer une définition avec le `where`, en utilisant la Curryfication, et avec une fonction anonyme.
 - (d) `rmEgaux :: [(Int,Int)] -> [(Int,Int)]` qui supprime tous les pairs (x, y) où $x = y$. Proposer une définition avec le `where`, en utilisant la Curryfication, et avec une fonction anonyme.
 - (e) Une version avec ZF-expressions de `rmChar` et comparer les résultats avec 2b.

Exercice 2 : (foldr et foldl)

Définir les fonctions suivantes de manière récursive en utilisant le filtrage par motifs, puis proposer pour chacune une définition équivalente en utilisant la fonction d’ordre supérieur `foldr`.

La signature de la fonction `foldr` est la suivante :

```
Prelude> :t foldr
foldr :: (a -> b -> b) -> b -> [a] -> b
```

- `produitRec, produitFold :: [Int] -> Int` qui calculent le produit des nombres entiers dans une liste.
- `etLogiqueRec, etLogiqueFold :: [Bool] -> Bool` qui vérifient si tous les termes dans la liste sont vrais.
- `concatRec, concatFold :: [[a]] -> [a]` qui font la concaténation d’une liste de listes (deux dimensions) en seule liste (une seule dimension).

```
Prelude> concatFold ["nous ", "formons ", "une ", "liste."]
"nous formons une liste."
```

- `rmCharsRec, rmCharsFold :: String -> String -> String` qui suppriment tous les caractères de la première chaîne qui apparaissent dans la seconde chaîne de caractères en réutilisant votre fonction `rmChar` de l'exercice précédent.
-

```
Prelude> rmCharsFold "ma" "programmation"
"progrtion"
```

Exercice 3 : (système de typage type)

Dans cette exercice, On s'intéresse à la décomposition en facteurs premiers des entiers naturels. Tout entier naturel $n > 2$ peut s'écrire de manière unique sous la forme :

$$n = p_1^{\alpha_1} \times p_2^{\alpha_2} \times \cdots \times p_k^{\alpha_k}$$

où p_i sont des nombres premiers tels que $p_1 < p_2 < \cdots < p_k$ et α_i sont des entiers naturels non nuls avec $1 \leq i \leq k$. Chaque décomposition est représentée par une liste de couples (facteur, exposant).

1. Définir dans votre fichier les nouveaux types **Facteur** et **Exposant**, qui sont des entiers, **Couple**, qui est un tuple constitué d'un **Facteur** et d'un **Exposant**, ainsi que **Decomposition**, qui est une liste de **Couple**.
2. Proposer deux définitions équivalentes de la fonction `(rep2int xs)` qui retourne l'entier associé à une décomposition.

```
> rep2int [] ==> 1
> rep2int [(2,3), (5,2)] ==> 200
> rep2int [(2,1), (5,1), (7,3)] ==> 3430
```

- (a) la première en utilisant une ZF-expression (listes en compréhension)
- (b) la seconde en utilisant une écriture récursive et le pattern-matching

Indication :

```
rep2intBis [] =
rep2intBis ((x,n):xs) =
```

3. Définir la fonction `(estPremier xs)` qui détermine si une décomposition représente un nombre premier (sans utiliser `rep2int`).

```
> estPremier [(2,3), (5,2)] ==> False
> estPremier [(2,1)] ==> True
```

4. Proposer deux définitions équivalentes de la fonction `(pgcd xs ys)` qui calcule le pgcd (Plus Grand Commun Diviseur) de 2 entiers représentés par leur décomposition (sans utiliser `rep2int`).

```
> pgcd [(2,3), (5,2)] [(2,1), (5,1), (7,3)] ==> [(2,1), (5,1)]
> pgcd [(2,3), (5,2)] [(2,2)] ==> [(2,2)]
> pgcd [(2,3), (5,2)] [(7,3)] ==> []
```

- (a) la première en utilisant une ZF-expression
- (b) la seconde en utilisant une écriture récursive et le pattern-matching

Indications

```
pgcdBis [] _ =
pgcdBis _ [] =
pgcdBis ((k1,d1):p1) ((k2,d2):p2)
| k1 == k2 =
|         =
|         =
```

5. Définir la fonction (`ppcm xs ys`) qui calcule le ppcm (Plus Petit Commun Multiple) de 2 entiers représentés par leur décomposition (sans utiliser `rep2int`).

```
> ppcm [(2,3), (5,2)] [(7,3)] ==> [(2,3), (5,2), (7,3)]
> ppcm [(2,3), (5,2)] [(2,2)] ==> [(2,3), (5,2)]
> ppcm [(2,3), (5,2)] [(2,1), (5,1), (7,3)] ==> [(2,3), (5,2), (7,3)]
```

Indication : on pourra s'inspirer de la définition de la fonction (`pgcdBis xs ys`)

6. Définir la fonction (`nbDiviseurs xs`) qui détermine le nombre de diviseurs du nombre entier dont la décomposition est `xs` (sans utiliser `rep2int`).

```
> nbDiviseurs [(2,1)] ==> 2
> nbDiviseurs [(3,2)] ==> 3
> nbDiviseurs [(2,1), (3,2)] ==> 6
> nbDiviseurs [(2,1), (3,2), (7,3)] ==> 24
```

Indications :

- Quel est le nombre de diviseurs du nombre a^n lorsque a est un nombre premier ?
- Si 2 nombres a et b sont premiers entre eux, quel est le lien entre le nombre de diviseurs de $(a \times b)$ et les nombres de diviseurs des nombres a et b ?
- En déduire la définition de la fonction `nbDiviseurs`.

7. On souhaite retrouver la liste des diviseurs d'un entier à partir de sa décomposition.

```
> diviseurs [(5,2)] ==> [1,5,25]
> diviseurs [(2,3), (5,2)] ==> [1,5,25,2,10,50,4,20,100,8,40,200]
> op (2,3) [1,5,25] ==> [1,5,25, 2,10,50, 4,20,100, 8,40,200]
```

- (a) Que fait la fonction (`op n ys`) définie ci-dessous ?

```
op :: Couple -> [Int] -> [Int]
op (x,n) ys = [(x^i) * y | i <- [0..n], y <- ys]
```

- (b) En déduire la définition de (`diviseurs xs`)

8. Premier exemple de traitement d'une liste infinie.

On définit la fonction suivante :

```
primes :: [Int]
primes = sieve [2 .. ]
where sieve (p:xs) = p : (sieve [x | x <- xs, mod x p > 0])
```

Évaluer les expressions suivantes et commenter les résultats obtenus :

```
> primes ==> ?
> take 10 primes ==> ?
> takeWhile (<= 100) primes ==> ?
> length (takeWhile (<= 10000) primes) ==> ?
```